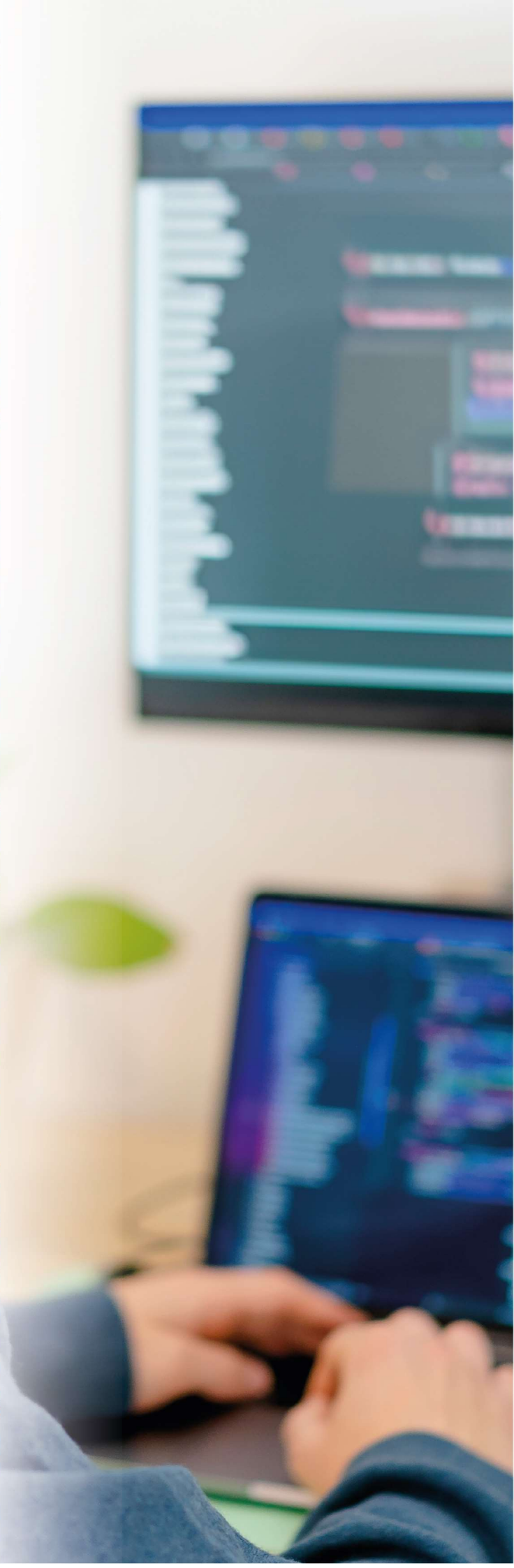
DESARROLLADOR DE APLICACIONES



Técnico Superior en Desarrollo de Aplicaciones Multiplataforma

0486 MP Acceso a Datos
UD1 Manejo de Ficheros

Índice

01. Acceso a un fichero. Clases asociadas

02. Gestión de flujo de datos

2.1. Clase FileWriter
2.2. Clase FileReader
2.3. BufferedReader y BufferedWriter
2.4. Clase FileOutputStream
2.5. Clase FileInputStream
2.6. RandomAccessFile
2.7. Ejemplo de uso de flujos

03. Bibliografía

01. Acceso a un fichero. Clases asociadas

El paquete java.io de Java, dispone de varias clases para dan soporte al trabajo con ficheros desde diferentes perspectivas:

- **Según el tipo de contenido:** ficheros de caracteres o de bytes (binarios).
- **Según el modo de acceso:** ficheros de acceso secuencial o aleatorio.

SEGÚN EL TIPO DE CONTENIDO:

- **Ficheros de caracteres (o de texto):** son aquellos ficheros creados exclusivamente con caracteres. Este tipo de ficheros, por lo tanto, pueden ser creados y visualizados utilizando cualquier editor que ofrezca el sistema operativo, como, por ejemplo, Notepad, Vi, Edit, etc.
- **Ficheros binarios (o de bytes):** son aquellos ficheros que no contienen caracteres reconocibles, sino que los bytes que contienen representan otra información: imágenes, música, vídeo, etc.
  Este tipo de archivos solo pueden ser abiertos por programas específicos que sepan cómo están organizados los bytes dentro del archivo, para así poder identificar la información que contiene.

SEGÚN EL MODO DE ACCESO:

- **Ficheros secuenciales:** la información se guarda como una secuencia continua de bytes (o caracteres), por lo que para acceder a un byte o carácter específico, primero se debe pasar por todos los anteriores.
- **Ficheros aleatorios:** se puede acceder directamente a una posición específica dentro del archivo sin tener que recorrer los datos previos. Un ejemplo de acceso aleatorio en programación es el uso de arrays.

Con independencia del tipo de fichero, Java define una clase dentro del paquete java.io que representa un archivo o directorio dentro de un sistema de ficheros: **clase File**.

- Un objeto de la clase File representa el nombre de un fichero o de un directorio existente en el sistema de ficheros.
- Los métodos de File permiten acceder a toda la información sobre las características del fichero o directorio.

EJEMPLO:

```
File f = new File ("proyecto\\libros.xml");
```

Si sobre este nuevo objeto File creado se aplica el siguiente código:

```
	if(f.exists()){
		System.out.println ("Nombre: " + f.getName() );
		System.out.println ("Directorio padre: " + f.getParent() );
		System.out.println ("Ruta relativa: " + f.getPath() );
		System.out.println ("Ruta absoluta: " + f.getAbsolutePath() );
	 }
```

El resultado será:

```
	Nombre: libros.xml
	Directorio padre: proyecto
	Ruta relativa: proyecto\libros.xml
	Ruta absoluta: C:\accesoDatos\proyecto\libros.xml
```

Os recomiendo ver el primer vídeo de este tema, 1-Objeto File en el que explico este ejercicio.

02. Gestión de flujo de datos

En Java, el acceso a ficheros es tratado como un flujo (stream) de información entre el programa y el fichero.

- Para comunicar un programa con cierta información externa (fichero) se usan flujos de información.
- Un flujo no es más que un objeto que hace de intermediario entre el programa y el origen o el destino de la información.

Java ofrece las siguientes clases:

- **FileReader**: implementa un flujo de caracteres que lee de un fichero de texto.
- **FileWriter**: implementa un flujo de caracteres que se escriben en un fichero de texto.

- **FileInputStream y FileOutputStream**: implementan flujos de lectura o escritura en ficheros binarios.
- **RandomAccessFile**: permite un acceso aleatorio a cualquier posición dentro del objeto.

Con independencia del tipo de flujo que se maneje, todos los accesos se hacen más o menos de la misma manera:

- **Para leer:**
  - Se abre un flujo desde un fichero.
  - Mientras haya información se lee la misma.
  - Una vez terminado, se cierra el flujo.
- **Para escribir:**
  - Se abre el flujo desde un fichero.
  - Mientras haya información se escribe en el flujo.
  - Una vez terminado, se cierra el flujo.

Los problemas de gestión del archivo (cómo se escribe en binario o cómo se hace cuando el acceso es secuencial o es aleatorio) quedan bajo la responsabilidad de la clase que implementa el flujo.

2.1. Clase FileWriter

El flujo FileWriter permite escribir caracteres en un fichero de modo secuencial. Sus constructores principales son:

FileWriter (String ruta, boolean añadir)
FileWriter (File fichero)

Donde:

- **ruta**: indica la localización del archivo en el sistema operativo.
- **añadir**: igual a true indica que el fichero se usa para añadir datos a un fichero ya existente.

El método más popular de FileWriter es el método write(), que puede aceptar un array de caracteres, pero también puede aceptar un string.

public void write (String str) throws IOException

2.2. Clase FileReader

El flujo FileReader permite leer caracteres desde un fichero de modo secuencial. Sus constructores principales son:

FileReader (String ruta)
FileReader (File fichero)

El fichero puede abrirse con una ruta de directorios o con un objeto de tipo File.

El método más popular de FileReader es el método read().

- Este método devuelve un número que si le hacemos un casting a char será legible para nosotros.
- Cuando se termina el fichero, el método read devuelve -1.

public int read () throws IOException

Os recomiendo ver el segundo vídeo de este tema, 2-FileWriter_FileReader en el que muestro en un ejemplo cómo escribir un archivo usando FileWriter y posteriormente leerlo usando FileReader.

2.3. BufferedReader y BufferedWriter

Estas clases tienen la misma función que FileReader y FileWriter, leer y escribir en ficheros, pero optimizan estas funciones.

Se crean igual que FileReader y FileWriter, pero como parámetro insertaremos un objeto FileReader y un objeto FileWriter respectivamente.

```
BufferedReader br = new BufferedReader(new FileReader("C:\\Libros.xml"));
BufferedWriter bw=new BufferedWriter(new FileWriter("C:\\Libros.xml"));
```

Las principales ventajas de los buffered son:

- BufferedReader nos permite leer una línea completa, en lugar de carácter a carácter como hacia FileReader. Cuando el fichero termina, devuelve null, no un -1 como en FileReader.
- Con BufferedWriter también podemos añadir una línea, como si pulsáramos un Enter.

Os recomiendo ver el tercer vídeo de este tema, 2-BufferedWriter_BufferedReader en el que muestro en un ejemplo cómo escribir un archivo usando BufferedWriter y posteriormente leerlo usando BufferedReader.

2.4. Clase FileOutputStream

El flujo FileOutputStream permite escribir bytes en un fichero de manera secuencial. Sus constructores tienen los mismos parámetros que los mostrados para FileWriter: el fichero puede ser abierto vacío o listo para añadirle datos a los que ya contenga. Además, hay que tener en cuenta que ya que este flujo está destinado a ficheros binarios (bytes) todas las escrituras se hacen a través de un buffer (array de bytes).

public void write (byte[ ] b) throws IOException

2.5. Clase FileInputStream

El flujo FileInputStream permite leer bytes en un fichero de manera secuencial.
Sus constructores tienen los mismos parámetros que los mostrados para FileReader.
El método más popular de FileInputStream es el método read():

public int read () throws IOException

2.6. RandomAccessFile

El flujo RandomAccessFile permite acceder directamente a cualquier posición dentro del fichero.

Proporciona dos constructores básicos:

RandomAccessFile (String ruta, String modo)
RandomAccessFile (File fichero, String modo)

Donde el parámetro modo determina para qué se abre el archivo:

- "**r**": sólo lectura o
- "**rw**": lectura y escritura.

2.7. Ejemplo de uso de flujos

De manera resumida el uso de flujos comparte los siguientes pasos:

SE ABRE EL FICHERO:

Se crea un objeto de la clase correspondiente al tipo de fichero que se quiere manejar y el modo de acceso.

De manera general la sintaxis para los objetos de tipo FileInputStrem, FileOutputStream, FileReader y FileWriter es:

TipoDefichero obj = new TipoDeFichero (ruta)

Donde ruta puede ser una cadena de texto con la ruta de acceso al fichero en el sistema operativo o puede ser un objeto de tipo File.

Para objetos RandomAccessFile es necesario indicar también el modo en el que se desea abrir el fichero: r (lectura) o rw (lectura y escritura).
RandomAcessFile obj = new RandomAcessFile (ruta, modo)

SE UTILIZA EL FICHERO:

Se utilizan los métodos específicos de cada clase para leer o escribir.

GESTIÓN DE EXCEPCIONES:

Todos los métodos que utilicen funcionalidad de java.io deben tener en su definición una cláusula throws IO Exception.

Los métodos de estas clases pueden lanzar excepciones de esta clase (o sus hijas) durante su ejecución, y dichas excepciones deben ser capturadas y debidamente gestionadas para evitar problemas.

SE CIERRA EL FICHERO Y SE DESTRUYE EL OBJETO:

Una vez se ha terminado de trabajar con el fichero lo recomendable es cerrarlo usando el método close() de cada clase:

```
obj.close();
```

EJEMPLO:

```
import java.io.FileWriter;
public void EscribeFicheroTexto() {

        //Crea el String con la cadena XML
        String texto =
                "<Libros><Libro><Titulo>El Quijote</Titulo></Libro></Libros>";

        //Guarda en la variable nombre el nombre del archivo que se creará.
        String nombre = "libros.xml";

        try{
                //Se crea un Nuevo objeto FileWriter
                FileWriter fichero = new FileWriter (nombre);

                //Se escribe el fichero
                fichero.write(texto + "\r\n");

                //Se cierra el fichero
                fichero.close();

        }catch (IOException ex){
                System.out.println("error al acceder al fichero");
        }
}
```

03. Bibliografía

Córcoles Tendero, J.E. (2013). Acceso a datos. Editorial Ra-Ma.

Ramos Martín, A. y Ramos Martín, MJ. (2016). Acceso a datos. Editorial Garceta.

Cortijo Bon, C.A. (2019). Acceso a datos. Editorial Síntesis.